

PROGRAMACIÓN III

Paradigma Imperativo vs. Paradigma Funcional en JavaScript (Node.js)

El objetivo de este apunte es comprender la transición conceptual desde el **pensamiento imperativo tradicional (C / C#)** hacia el **paradigma funcional** y la **inmutabilidad en JavaScript moderno (ES6+)**. Aprenderemos a manipular colecciones mediante métodos declarativos iterativos: `map()`, `filter()` y `reduce()`.

1. Introducción: Pensamiento Imperativo vs. Paradigma Funcional

Al abordar el desarrollo en JavaScript moderno, uno de los principales desafíos radica en cambiar la forma de razonar la resolución de problemas. En entornos imperativos (C / C# tradicional), el enfoque está centrado en **cómo** ejecutar una tarea paso a paso. En el paradigma funcional, el enfoque se desplaza hacia **qué** transformación deseamos aplicar sobre los datos.

Concepto	Pensamiento Imperativo (C / C#)	Paradigma Funcional (JS / ES6+)
Enfoque principal	How to do: Pasos detallados y secuencia de órdenes.	What to do: Declarativo, enfocado en transformaciones.
Estado de datos	Mutable: Las variables cambian de valor explícitamente.	Inmutable: Se devuelven nuevas estructuras sin alterar las originales.
Control de flujo	Estructuras manuales: for, while, condicionales.	Métodos declarativos: map, filter, reduce, find.
Tratamiento de funciones	Métodos subordinados a clases o estructuras de bloque.	Ciudadanos de primera clase (First-class functions / Callbacks).

2. El Método `Array.prototype.map()`

El método `map()` toma un arreglo de entrada, aplica una función transformadora sobre cada uno de sus elementos y devuelve un nuevo arreglo con la misma cantidad de elementos, sin modificar la estructura de origen.

 **Importante en MAP:** `map()` siempre genera un nuevo arreglo de exactamente la misma longitud que el arreglo de entrada. Nunca debe usarse para filtrar elementos ni para ejecutar efectos secundarios (side-effects).

2.1 Firma sintáctica de map()

```
const nuevoArreglo = arregloOriginal.map((elemento, indice, arreglo) => {  
  return elementoTransformado;  
});
```

2.2 Análisis detallado de una proyección: Desglose de código

Analicemos minuciosamente la siguiente línea de código para comprender qué sucede internamente:

```
const nombres = estudiantes.map(estudiante => estudiante.nombre);
```

- **const nombres = ...:** Declara una constante donde se almacenará la referencia al nuevo arreglo generado en memoria.
- **estudiantes.map(...):** Invoca el iterador. Actúa como un motor de recorrido que visita cada posición en orden secuencial.
- **estudiante => ...:** Parámetro del callback. Representa el elemento actual que se está procesando en cada vuelta.
- **... => estudiante.nombre:** Retorno implícito de la arrow function. Extrae la propiedad .nombre y la deposita en la misma posición del nuevo arreglo.

2.3 Evolución de sintaxis: De C# a JavaScript Idiomático

Para comprender el nivel de abstracción alcanzado en JS, observemos cuatro formas equivalentes de resolver la extracción de nombres:

```
// Opción A: Imperativo tradicional (Estilo C# / C)  
const nombres = [];  
for (let i = 0; i < estudiantes.length; i++) {  
  nombres.push(estudiantes[i].nombre);  
}  
  
// Opción B: .map() con función anónima tradicional  
const nombres = estudiantes.map(function(estudiante) {  
  return estudiante.nombre;  
});  
  
// Opción C: .map() con Arrow Function explícita  
const nombres = estudiantes.map((estudiante) => {  
  return estudiante.nombre;  
});  
  
// Opción D: JS Idiomático (Retorno implícito en una sola línea)  
const nombres = estudiantes.map(estudiante => estudiante.nombre);
```

3. Filtrado y Reducción: filter() y reduce()

3.1 Array.prototype.filter()

Evalúa un predicado booleano (una función que retorna true o false) sobre cada elemento del arreglo. Devuelve un nuevo arreglo únicamente con los elementos que cumplieron la condición.

```
// Ejemplo de uso directo de filter()
const productos = [
  { id: 1, nombre: 'Teclado', precio: 80, stock: true },
  { id: 2, nombre: 'Mouse', precio: 25, stock: false },
  { id: 3, nombre: 'Monitor', precio: 200, stock: true }
];

// Selecciona productos con stock disponible
const disponibles = productos.filter(p => p.stock);
```

3.2 Array.prototype.reduce()

Procesa los elementos de una colección para combinarlos y reducirlos a un único valor resultante (que puede ser un número, un objeto acumulador, un string o una nueva estructura).

```
// Ejemplo de acumulación con reduce()
const carrito = [
  { producto: 'Laptop', precio: 1200 },
  { producto: 'Funda', precio: 30 }
];

// El 0 final representa el valor inicial del acumulador
const total = carrito.reduce((acum, item) => acum + item.precio, 0);
```

4. Composición de Operaciones: Pipeline Funcional (Chaining)

En la práctica profesional backend/frontend, estos métodos se encadenan formando un pipeline o flujo de datos sin necesidad de almacenar variables intermedias mutables.

```
// Ejemplo de Chaining: Transacciones de ventas
const transacciones = [
  { id: 'T1', monto: 1500, estado: 'APROBADA' },
  { id: 'T2', monto: 3000, estado: 'RECHAZADA' },
  { id: 'T3', monto: 800, estado: 'APROBADA' }
];

// PIPELINE: 1. Filtrar Aprobadas -> 2. Aplicar 10% descuento -> 3. Sumar total
const ingresoNeto = transacciones
  .filter(t => t.estado === 'APROBADA')
  .map(t => t.monto * 0.90)
  .reduce((acum, monto) => acum + monto, 0);

console.log(`Ingreso neto procesado: ${ingresoNeto}`);
```

5. Scripts Ejecutables para Entorno Node.js

A continuación se proveen los módulos JS listos para ejecutar directamente mediante la consola con Node.js:

Script 01: Comparativa Imperativa vs Funcional

```
// 01-comparativa-iva.js
const precios = [100, 250, 500];

// Imperativo (for)
const preciosConIVAImperativo = [];
for (let i = 0; i < precios.length; i++) {
  preciosConIVAImperativo.push(precios[i] * 1.21);
}

// Funcional (map)
const preciosConIVAFuncional = precios.map(precio => precio * 1.21);

console.log('Entrada:', precios);
console.log('Salida Funcional:', preciosConIVAFuncional);
```

Script 02: Transformación de Objetos y Spread Operator

```
// 02-transformar-objetos.js
const estudiantes = [
  { id: 1, nombre: 'Ana', nota: 8 },
  { id: 2, nombre: 'Luis', nota: 5 }
];

// Enriquecimiento de datos inmutable
const estudiantesConEstado = estudiantes.map(e => ({
  ...e,
  aprobado: e.nota >= 6
}));

console.table(estudiantesConEstado);
```

6. Guía de Práctica y Ejercicios para Clase

A continuación se adjunta la guía de actividades prácticas propuestas para los estudiantes:

Ejercicio 1: Procesamiento de Calificaciones (map)

A partir de un listado de estudiantes con sus notas numéricas, generar un nuevo arreglo de objetos añadiendo la condición ('APROBADO' si nota >= 6, sino 'RECURSA').

```
const estudiantes = [
  { nombre: 'Ariel', nota: 8 },
  { nombre: 'Brenda', nota: 4 }
];

const resultado = estudiantes.map(e => ({
  ...e,
  condicion: e.nota >= 6 ? 'APROBADO' : 'RECURSA'
}));
```

Ejercicio 2: Control de Inventario (filter + reduce)

Filtrar productos activos con cantidad > 0 y obtener el valor total acumulado en dinero de dicho inventario.

```
const productos = [
  { id: 101, nombre: 'Monitor', precio: 150000, cantidad: 5, stock: true },
  { id: 102, nombre: 'Teclado', precio: 25000, cantidad: 0, stock: true }
];

const valorTotal = productos
  .filter(p => p.stock && p.cantidad > 0)
  .reduce((acum, p) => acum + (p.precio * p.cantidad), 0);
```

Ejercicio 3: Reporte de Horas Laborales (filter + map + reduce)

Filtrar registros de la categoría 'DESARROLLO' con al menos 4 horas, convertir las horas a monto (\$12.500/hora) y calcular el total general a liquidar.

```
const registros = [
  { empleado: 'Lucía', horas: 8, categoria: 'DESARROLLO' },
  { empleado: 'Marcos', horas: 2, categoria: 'DESARROLLO' }
];

const VALOR_HORA = 12500;
const totalLiquidar = registros
  .filter(r => r.categoria === 'DESARROLLO' && r.horas >= 4)
  .map(r => r.horas * VALOR_HORA)
  .reduce((total, monto) => total + monto, 0);
```